doi :10.3969/j.issn.1671-1122.2025.12.002

大语言模型引导的协议模糊测试技术研究

杨立群[1,2,3]，李镇[1]，韦超仁[1]，闫治敏[1]，仇勇鑫[1]

（1. 北京航空航天大学网络空间安全学院，北京 100191；2. 中国民用航空飞行学院民航安全工程学院，广汉 618307；3. 四川省全电通航飞行器关键技术工程研究中心，广汉 618307）

摘　要：网络协议软件漏洞频发，严重威胁网络空间安全。以 AFLNet 为代表的灰盒协议模糊测试工具通过引入覆盖率反馈与状态建模机制提升了漏洞挖掘能力，但受限于“语义屏障”，此类工具难以理解协议的语法结构与上下文逻辑，测试效率较低。近年来，大语言模型在语义建模、上下文推理与代码生成等任务中展现出强大的泛化与理解能力，为打破这一屏障提供了关键技术路径。文章提出了一种由大语言模型引导的协议模糊测试框架（LLMProFuzz，LPF），在以下 3 个层面克服传统方法的局限性：一是利用少样本提示工程自动提取协议语法模板；二是基于历史漏洞特征设计种子富集机制，生成覆盖边界场景和异常逻辑的高价值初始用例；三是引入结构感知的变异位置选择策略，提高有效测试用例的生成比例。在 HTTP、FTP、RTSP 等典型协议栈中的实验结果表明，LPF 在代码覆盖率、状态覆盖率和测试效率等方面均显著优于 AFLNet 与 StateAFL 等基准工具。

关键词：大语言模型；网络协议；模糊测试；结构感知变异；提示工程

中图分类号：TP309　**文献标志码：**A　**文章编号：**1671-1122（2025）12-1847-16

中文引用格式：杨立群，李镇，韦超仁，等. 大语言模型引导的协议模糊测试技术研究 [J]. 信息网络安全，2025，25（12）：1847–1862.

英文引用格式：YANG Liqun, LI Zhen, WEI Chaoren, et al. Research on Protocol Fuzzing Technology Guided by Large Language Models[J]. Netinfo Security, 2025, 25(12): 1847–1862.

Research on Protocol Fuzzing Technology Guided by Large Language Models

YANG Liqun[1,2,3], LI Zhen[1], WEI Chaoren[1], YAN Zhimin[1], QIU Yongxin[1]

(1. *School of Cyber Science and Technology*, *Beihang University*, *Beijing* 100191, *China*; 2. *School of Civil Aviation Safety Engineering*, *Civil Aviation Flight University of China*, *Guanghan* 618307, *China*; 3. *Engineering Research Center of Key Technologies for All-Electric General Aviation Aircraft of Sichuan Province*, *Guanghan* 618307, *China*)

Abstract: Security vulnerabilities in network protocol software occur frequently and pose serious threats to cyberspace security. Gray-box protocol fuzzing tools, such as

收稿日期：2025-08-23

基金项目：国家自然科学基金 [62302025,62572027,U2333205]；四川省全电通航飞行器关键技术工程研究中心开放课题 [CAFUC 202401KF003]

作者简介：杨立群（1990—），男，河北，副教授，博士，CCF 会员，主要研究方向为网络信息安全、工业互联网和工控安全；李镇（1995—），男，山东，硕士研究生，主要研究方向为网络信息安全、模糊测试；韦超仁（2002—），男，四川，硕士研究生，主要研究方向为模糊测试、大语言模型；闫治敏（2003—），男，内蒙古，硕士研究生，主要研究方向为大语言模型、模糊测试；仇勇鑫（2001—），男，山西，硕士研究生，主要研究方向为模糊测试。

通信作者：杨立群 lqyang@buaa.edu.cn

万方数据

AFLNet, have improved vulnerability detection by introducing coverage feedback and state modeling mechanisms. However, constrained by a persistent "semantic barrier", these tools struggle to comprehend protocol syntax structures and contextual logic, resulting in limited testing efficiency. In recent years, large language models have demonstrated exceptional generalization and comprehension capabilities in tasks such as semantic modeling, contextual reasoning, and code generation, providing a promising pathway to overcome this barrier. This paper proposed LPF (LLMProFuzz), a protocol fuzzing framework guided by large language models, which addressed the limitations of traditional methods from three perspectives: firstly, automatically extracting protocol syntax templates through few-shot prompt engineering; secondly, designing a seed enrichment mechanism based on historical vulnerability characteristics to generate high-value initial cases that cover boundary and exceptional scenarios; thirdly, introducing a structure-aware mutation location selection strategy to increase the proportion of effective test cases. Experimental results on representative protocol stacks, including HTTP, FTP, and RTSP, demonstrate that LPF significantly outperforms baseline tools such as AFLNet and StateAFL in terms of code coverage, state coverage, and testing efficiency.

Key words: large language models; network protocol; fuzzing; structure-aware mutation; prompt engineering

0 引言

随着网络技术的不断演进，网络协议已成为现代信息系统中不可或缺的基础组件，其安全性直接关系到国家关键基础设施的稳定运行、企业信息系统的安全防护以及公众数据的隐私保障。然而，协议实现逻辑的不断复杂化与漏洞检测机制的滞后性为攻击者创造了众多可乘之机。近年来，网络协议漏洞相关的安全事件频发，造成服务中断、数据泄露、远程控制等严重后果，这些都凸显了网络协议漏洞挖掘工作的重要性和紧迫性[1]。2023年发现的"Rapid Reset"漏洞（CVE-2023-44487）主要影响HTTP/2协议服务器，攻击者可以利用该协议的快速重置机制发起分布式拒绝服务（Distributed Denial of Service，DDoS）攻击，导致Google、Amazon和Microsoft等云平台面临前所未有的风险。2024年披露的"RegreSSHion"漏洞（CVE-2024-6387）允许未授权攻击者通过构造特殊SSH报文以root身份执行任意代码，影响了全球超过1400万台SSH服务器。此外，通用漏洞披露（Common Vulnerabilities and Exposures，CVE）平台数据表明，近十年全球新增漏洞数量持续攀升，年增长幅度更是居高不下，其中，2024年披露的漏洞总数已超过40000个，再创历史新高。这些案例与数据均表明，网络协议安全已成为网络空间安全治理中的关键挑战[2]。

协议模糊测试（Protocol Fuzzing）作为一种主流的自动化漏洞挖掘方法，在学术界和工业界受到广泛关注[3]。该技术经历了从黑盒测试到白盒测试再到灰盒测试的演变[4]。基于黑盒的模糊测试方法（如SPIKE、Peach等）依赖人工或规则驱动，缺乏对运行状态的感知能力；基于白盒的模糊测试方法（如SAGE）通过符号执行实现精确路径探索，但因其计算开销巨大，难以适用于大规模系统[5,6]。2020年，PHAM[7]等人提出了AFLNet，将覆盖率引导的灰盒模糊测试策略引入有状态网络协议测试，标志着协议模糊测试进入了"覆盖率驱动"的智能化时代。然而，AFLNet在实践中仍面临以下三大挑战。

1）协议语法与语义感知能力欠缺

AFLNet在测试用例生成与种子变异过程中缺乏对协议语法和上下文语义的深入理解，导致测试工具缺乏语法指导[8]。

2）初始种子覆盖面不足

在测试执行过程中，AFLNet依赖的初始种子数量有限，难以覆盖协议交互的边界路径与异常场景，初

万方数据

始种子库的消息类型较少，限制了模糊测试的探索能力，导致测试容易陷入浅层路径的重复探索。

3）变异策略盲目且低效

AFLNet采用随机化的位翻转、字节替换等固定算子进行种子变异，这种缺乏结构和语义约束的变异策略容易破坏测试用例的格式合法性，导致测试用例缺乏实际价值，严重制约测试效率的提升。

以上问题的根源在于测试工具缺乏对协议语法和语义的理解，因此，仅通过优化覆盖率反馈算法或改进变异算子等方式已难以实现测试能力的质变。要突破这一瓶颈，模糊测试技术亟需从“浅层指标引导的暴力破解”范式，演进为“深层理解驱动的智能测试”范式。

近年来，人工智能特别是大语言模型（Large Language Models，LLMs）技术发展迅猛，以ChatGPT系列为代表的LLMs能够从输入中识别上下文关系、抽取语法规则并生成语义合法的代码片段，这与协议模糊测试对输入语义建模和结构感知的需求高度契合，为解决协议模糊测试中的“语义鸿沟”问题提供了新思路。尽管LLMs广泛应用于程序测试、代码生成、多模态融合等方面，但将其系统地融入有状态协议模糊测试中，构建一套完整的LLMs驱动的模糊测试机制，解决协议语法有效提取、种子高质量生成与变异效率改善等核心问题，仍是一个具有挑战性且有研究价值的课题。为此，本文提出了一种由大语言模型引导的协议模糊测试框架（LLMProFuzz，LPF）。该框架构建了一种由大语言模型赋能的灰盒协议模糊测试机制，将LLMs的语义理解与生成能力深度融入预处理（协议语法提取）、初始化（基于漏洞特征的种子富集）和测试执行（结构感知变异）3个核心阶段，从而有效提升了协议模糊测试的智能化水平。本文的主要贡献如下。

1）提出了一种基于LLMs多轮对话的协议语法提取方法，结合LLMs对自然语言的理解和生成能力，根据协议规范文件，自动生成高精度协议语法模板，采用多轮交互和一致性检查方法筛选有效语法项，为结构化测试用例生成和语义约束变异提供支撑。

2）设计了一种基于历史漏洞特征的种子富集策略，通过提示词引导LLMs生成包含已知漏洞触发模式的高价值初始种子，有效提升了种子库对边界路径与异常场景的覆盖能力。

3）提出了一种基于结构感知的种子变异优化策略，基于协议语法结构，指导模糊测试器选择合适的种子变异位置，从而提升测试用例变异的有效性。

4）在多个典型协议测试场景下对LPF进行对比实验，结果表明，LPF较AFLNet和StateAFL的代码覆盖率分别提升了4.59%和5.44%，状态覆盖率分别提升了19.78%和3.32%。此外，LPF在崩溃触发速度上也显著优于现有工具。

1 相关工作

1.1 AFLNet 工具

AFLNet是现代协议模糊测试领域的一项里程碑式工作，它将覆盖率引导的灰盒模糊测试思想从文件输入型程序扩展到了有状态的网络协议程序[9]。AFLNet作为基于遗传算法的高效模糊测试工具，其核心与AFL（American Fuzzy Lop）类似[10]。两者均通过对被测程序进行插桩来实时追踪代码执行路径，当变异输入触发新路径时，将其保留至种子队列并优先变异，以此高效探索程序代码空间。不同的是，AFLNet通过调用网络接口，从响应报文中提取状态码，利用服务器响应码来构建状态模型，引导模糊测试的执行过程。此外，AFLNet沿用了AFL的fork-server机制，显著降低了重复执行被测程序时的启动开销，提升了测试吞吐率。AFLNet原理如图1所示。

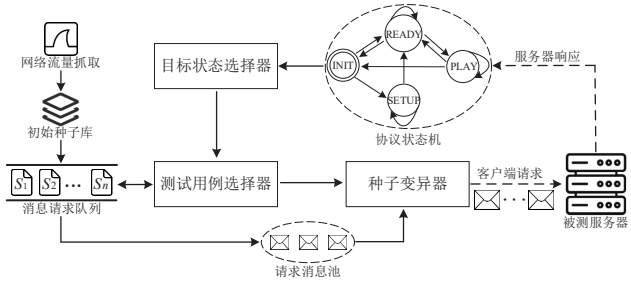


图 1 AFLNet 原理图

万方数据

具体来说，AFLNet在AFL的核心框架之上进行了以下3个方面的扩展。

1）网络通信模块

AFLNet增加了一个网络通信模块，使其能够扮演通信客户端的角色，通过TCP或UDP协议向待测试的服务器程序发送测试用例（即协议消息序列）。这使得模糊测试器能够与网络服务进行真实的交互。

2）协议状态追踪

对于有状态网络协议（如FTP、RTSP等），服务器的行为不仅取决于当前接收的消息，还依赖之前的交互历史信息。AFLNet通过监控服务器返回的响应码序列（如HTTP的“200 OK”和“404 Not Found”）对服务器的状态抽象建模。在测试过程中，一旦某个测试用例引导服务器到达了一个新状态，AFLNet便会将该测试用例视为高价值种子并纳入队列保留。这种基于状态反馈的机制能有效引导模糊测试深入探索协议的状态空间。

3）消息级变异策略

除了AFL原有的字节级变异策略外，AFLNet还引入了针对协议消息的变异策略，允许对消息序列中的单个消息进行独立的变异、删除或重复，以更好地模拟协议交互中可能出现的异常。

基于以上扩展，AFLNet能够系统性地对有状态网络协议进行模糊测试，并成功发现了多个知名软件（如LightFTP、Live555）中的安全漏洞。为提升测试效率，后续工作（如SNPSFuzzer[11]和SnapFuzz[12]）通过引入快照（Snapshot）技术避免了重复的网络重连，而NSFuzz[13]则通过识别网络事件循环消除了不必要的等待时间。这些方法虽然能在一定程度上提高模糊测试的吞吐率，但在初始用例获取和变异策略选择等方面仍存在较大的优化空间[14]，这也是本文工作的主要动机。

1.2 协议状态机

网络协议状态机（Protocol State Machine，PSM）是一种用于建模网络通信过程中状态变化行为的形式化工具，通过有限状态集合与明确定义的状态转移规则，描述协议实体在接收到消息或触发事件后如何从一个状态转移到另一个状态[15]。PSM在协议实现、模糊测试、形式验证和漏洞分析等领域具有广泛的应用价值。一般地，PSM可定义为一个五元组，如公式（1）所示。

$$M = (S, \Sigma, \delta, s_0, F) \tag{1}$$

其中，S表示状态集合，包含协议运行过程中的所有可能状态；Σ为输入符号集合，表示协议实体可能接收到的消息或事件；$\delta: S \times \Sigma \rightarrow S$是状态转移函数，定义在给定状态下接收特定输入后转移到的新状态；$s_0 \in S$为初始状态；$F \subseteq S$为终止状态集合，表示协议交互可能的结束状态。

在实际网络协议设计中，状态机主要用于规范通信流程，限制非法输入和异常行为。客户端与服务器在协议交互过程中依据状态机模型进行状态推进，从而实现连接建立、认证协商、数据传输、连接释放等功能。例如，在FTP协议中，服务器状态通常依次经历初始连接（INIT）、用户身份验证（AUTH_WAIT）、命令处理（COMMAND）、数据传输（TRANSFER）与连接关闭（CLOSE）等状态。协议状态机可以使用有向图建模，其中，节点表示状态，边表示状态之间的合法转移关系。设状态数为$|S|=n$，则理论上最多存在$n(n-1)$条有向状态迁移边，表示任意两个状态之间均可达。然而，在实际协议设计中，为了保障安全性与可控性，会显式约束状态之间的转移，仅保留合法路径。协议交互过程可抽象为一条状态路径，如公式（2）所示，每一步转移均满足公式（3），其中$a_i \in \Sigma$。

$$P=(s_0,s_1,\cdots,s_k) \tag{2}$$

$$s_{i+1}=\delta(s_i,a_i) \tag{3}$$

该路径可用于形式化分析协议的状态覆盖率、死状态、不可达状态及冗余状态等，是协议验证与模糊测试的重要基础。

状态机建模不仅有助于协议规范的编写与实现，更能在模糊测试技术中发挥重要作用。传统模糊测试

万方数据

由于缺乏协议状态感知能力，容易生成违反状态机逻辑的非法消息，导致被测系统提前关闭连接，状态覆盖率较低[16]。状态感知的模糊测试工具则依赖协议状态机信息，从合法的初始状态出发，逐步推进状态，生成与状态一致的有效消息序列，从而大幅提升测试效率与漏洞触发概率。

1.3 主流网络协议服务器软件

网络协议的实现依赖于特定的服务器软件，这些软件不仅要完成协议规范的报文解析与处理，还要承担实际网络环境中的功能执行与逻辑调度[17]。为方便理解面向协议服务器的模糊测试场景，本文针对流媒体（RTSP）、文件传输（FTP）、Web服务（HTTP）、电子邮件（SMTP）和实时通信（SIP）等典型协议，选取了以下5款具有代表性的开源服务器软件进行简要介绍。

1）Live555

基于C++开发的流媒体框架，完整实现了RTSP、RTP、RTCP及SIP协议。核心组件Live555 Media Server支持命令行接口，采用事件驱动模型与模块化架构，具备跨平台特性，广泛应用于IP摄像头、嵌入式视频终端及媒体代理服务器等场景。

2）ProFTPD

功能完备的FTP服务器，支持类Unix与Windows系统。其配置语法与Apache兼容，提供虚拟主机、目录级访问控制（.ftpaccess）、SSL/TLS加密及匿名访问等高级功能。

3）Lighttpd

面向高并发与资源受限环境的Web服务器，完整实现HTTP/1.1协议，采用单进程异步事件驱动架构，显著降低内存开销并提升吞吐性能，广泛部署于物联网网关、嵌入式平台及轻量级Web服务。

4）Exim

一款适用于类Unix系统的邮件传输代理（MTA），可实现SMTP/LMTP协议及TLS加密、身份验证、垃圾邮件过滤等扩展功能，为电子邮件系统提供高度灵活的传输控制能力。

5）Kamailio

基于SIP协议的高性能信令服务器，遵循RFC3261标准并扩展支持呼叫路由、负载均衡、TLS加密及WebRTC，广泛应用于电信级VoIP网络与企业统一通信平台。

选择以上主流开源软件作为测试目标，能够有效测试LPF框架对不同类型协议模糊测试的泛化能力。此外，这些软件拥有公开的漏洞历史信息和活跃的开发者社区，便于对发现的崩溃（Crash）进行验证和定性，从而更客观地评价本文方法在真实漏洞挖掘场景下的实际效果。

1.4 LLMs 及其在模糊测试中的应用

LLMs是基于海量文本数据训练的深度学习模型，其核心技术基础是2017年由VASWANI[18]等人提出的Transformer架构。与传统的循环神经网络（Recurrent Neural Network，RNN）或卷积神经网络（Convolutional Neural Network，CNN）不同，Transformer通过其创新的自注意力机制（Self-Attention）能够并行处理序列中的所有元素，并高效地捕捉长距离语义依赖关系。这一架构突破使得训练拥有数千亿甚至万亿级别参数的模型成为可能。以GPT系列、DeepSeek、Gemini等为代表的LLMs，通过在海量文本和代码语料上进行预训练，学习到了丰富的语言规律、事实知识和推理能力。

提示工程（Prompt Engineering）是与LLMs交互以引导其生成期望输出的关键技术[19]。一个精心设计的提示词（Prompt）能够有效激活LLMs的内部知识库，并指导其完成特定任务。在模糊测试场景中，提示工程主要用于提取协议语法、生成高质量的测试用例以及优化变异策略。少样本学习（Few-Shot Learning）是提示工程中的一种高级技巧，通过在提示中提供多个具体的“输入—输出”对作为示例，让模型在不进行参数更新的情况下，快速适应新任务的格式和要求。

近年来，研究人员积极探索LLMs在模糊测试领域中的应用，并取得了一系列令人瞩目的进展[20]。在

万方数据

基于生成的模糊测试工具中，Codamosa[21]将LLMs应用于Python模块的测试用例生成，通过模型自动推导函数的参数约束与异常处理分支，其代码覆盖率较人工编写的测试用例提升了25%。为测试深度学习框架API的安全性，TitanFuzz[22]利用LLMs生成高维张量输入与异常操作序列，在TensorFlow 2.12版本中发现了12个未知的安全漏洞。FuzzGPT[23]专注于生成边界测试用例，通过精心设计的提示词引导LLMs模拟协议交互过程中的极端场景（如并发请求冲突、服务器资源耗尽），在HTTP/3协议软件中成功挖掘到了3个零日漏洞。ECG[24]利用LLMs生成可调用目标内核API的C语言测试代码，并通过迭代编译反馈生成有效规约，实现高语义感知的测试用例构造，其代码覆盖率平均提升了16.02%。5GPT[25]融合5G领域安全属性、历史CVE信息与思维链提示方法，通过多轮交互和双层筛选机制，生成高质量测试用例，成功识别了47个潜在漏洞。RESTLess[26]通过融合LLMs和规约增强技术来生成复杂且语义合理的长请求序列，提升了API模糊测试的语义深度和状态覆盖率，并发现了38个漏洞。LLMIF[27]针对IoT协议（如Zigbee），结合提示和领域增强实现协议种子自动生成，使代码覆盖率提升了55%。

同时，基于变异的模糊测试工具也从LLMs中获益匪浅。FuzzCoder[28]开发了一种基于微调LLMs的字节级变异策略，能够从历史攻击样本中学习模式，提升字节序列上的脆弱性发现效率。BertRLFuzzer[29]结合了BERT模型与强化学习技术，动态优化Web漏洞的探测策略，使其在SQL注入与XSS漏洞上的发现率较传统方法提高了27%。WhiteFox[30]则将LLMs与白盒分析技术结合，通过模型理解编译器的中间表示来指导变异，显著提升了对编译器漏洞的发现能力。EOM[31]等人将覆盖率反馈与LLMs输出回路嵌入变异流程，通过强化学习加权改写JavaScript引擎测试输入，实现了漏洞发现与覆盖率的显著提升。此外，Fuzz4All[32]创新性地融合了提示工程与变异策略，利用LLMs从文档中提取简洁的提示，从而生成兼具多样性并且结构合规的测试代码片段。

以上研究表明，LLMs正逐步从辅助生成工具演进为测试策略的核心组成部分，基于LLMs的模糊测试工具能够显著提升测试效率与漏洞触发能力。然而，将LLMs应用于协议模糊测试仍面临一些挑战，包括如何保证语法提取的精度、如何降低模型交互的性能开销，以及如何利用LLMs指导变异策略等，这些都是本文需要着力解决的问题。

2 设计与实现

为了解决传统协议模糊测试的挑战，本文设计并实现了LPF（LLMProFuzz）框架。该框架基于AFLNet进行扩展，通过模块化设计，将LLMs的语义理解能力集成到灰盒模糊测试流程中。在AFLNet的基础上，LPF主要进行了以下3个部分的改进：1）基于LLMs的协议语法提取模块实现了协议语法信息的自动化提取；2）基于历史漏洞的种子富集模块扩展了初始种子的多样性及语义覆盖面；3）基于结构感知的变异策略优化方法提高了种子变异的质量和测试效率。

2.1 框架概述

LPF在继承AFLNet框架的状态追踪（用于识别协议状态转移）和代码覆盖率反馈（用于引导探索新路径）两大核心机制的基础上，引入了协议语法提取模块、种子富集模块和结构感知变异模块3个关键增强模块，从预处理、输入生成到变异执行3个层面系统性优化测试流程，其整体架构如图2所示。

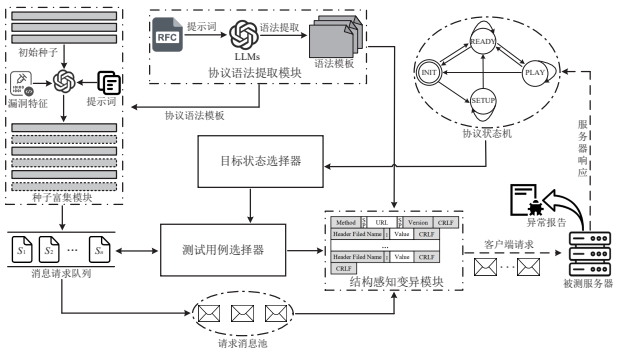


图 2 LPF 整体架构

协议语法提取模块主要工作在模糊测试的预处理

万方数据

阶段，在测试开始前调用LLMs解析协议文档或流量数据，生成结构化的协议语法模板库，为后续测试提供协议语法信息的先验知识；种子富集模块工作在模糊测试的初始化阶段，利用LLMs对历史漏洞触发模式的先验知识，生成融合了漏洞特征的初始测试用例，以扩展和优化初始种子库；结构感知变异模块结合语法模板对种子进行结构感知变异，取代传统模糊测试框架的随机变异逻辑，提升有效测试用例的生成比例，进而提升测试效率。LPF的3个核心引擎与AFLNet原有的种子队列、变异器、状态追踪器和反馈机制紧密集成，通过共享内存和消息队列进行通信，形成一个智能化的闭环反馈系统。

2.2 基于 LLMs 的协议语法提取

协议语法模板的精准提取是生成高质量测试用例的基础。为此，本文提出了一种基于LLMs的自动化协议语法提取方法，其核心在于利用少样本提示引导LLMs生成结构化的协议语法模板。具体流程如下。

1）提示词设计

在分析RFC标准规范文件、目标协议实现代码及采集的真实流量等多源文件的基础上，本文在提示词中引入协议版本关键词（如“RTSP”“RFC2326”“Live555”），旨在引导LLMs将知识检索范围限定于相关性较强的信息源，避免LLMs的回答过于发散。然后，采用少样本提示，即在提示中嵌入2~3个格式化良好的语法模板示例（如“DESCRIBE: ...”或“PLAY: ...”），让LLMs现场学习这些样例并“记住”回答格式，这将避免LLMs微调所需的复杂过程，如图3所示。

2）多轮交互与一致性检查

由于协议定义庞杂，单轮交互难以覆盖全部消息类型，本文设计了一种多轮交互机制，并将前一轮的对话内容作为上下文，以防止LLMs重复已有内容。本阶段采用主提示和补充提示两类提示模板。其中，主提示用于首次交互，引导模型生成主要消息模板；补充提示用于后续交互，并附带指令“only list new message types”以控制输出范围。此外，为解决采样生成的随机性问题，在多轮交互采样的基础上，本文基于一致性阈值筛选最终结果。具体来说，若某请求消息类型在总迭代次数中出现频次达2/3及以上（例如，30轮实验中出现不少于20次），则认定为有效语法并纳入最终模板库。该策略既能有效剔除偶发“幻觉”，又能保留低频但真实的字段。

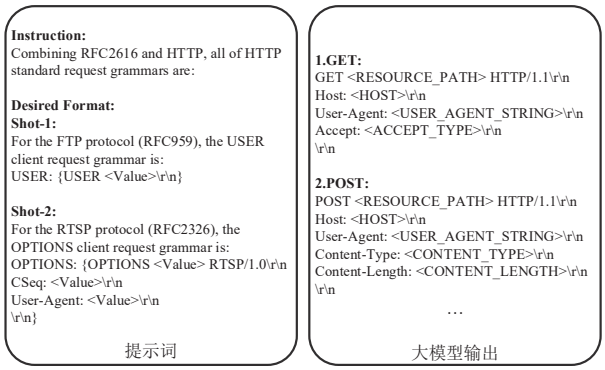


图 3 协议语法提取提示词

3）模板处理与数据结构

多轮交互的每次响应会经过自动解析，并剥离非结构描述信息，提取“消息类型：字段列表”内容。使用预定义分隔符（如“:”“\r\n”）对响应进行解析，将模板存储为JSON格式，键为消息类型（如“DESCRIBE”），值为字段结构对象，包括字段名称、正则表达式、位置等信息。这些模板被存入grammar_templates字典中，并导出为JSON形式的离线语法库，供后续消息级变异与语义校验调用。

算法1展示了基于LLMs自动提取协议语法模板的完整流程。

算法1 协议语法模板提取算法

输入：protocol_name
输出：msg_types_set
1. templates_prompt ← ConsPrompt(protocol_name)
2. for iter ← 0 to CONSISTENCY_COUNT do
3. grammar_list ← Init consistency_table
4. templates_ans ← ChatLLM(templates_prompt)
5. remaining_prompt ← RemPrompt(protocol_name)
6. remaining_ans ← ChatLLM(remaining_prompt)
7. comb_templates ← templates_ans ∪ remaining_ans
8. grammar_list ← extract_grammars(comb_templates)
9. for jobj ∈ grammar_list do

万方数据

```
10.     header ← jobj[0]
11.     field_table ← consistency_table[header]
12.     for i ← 1 to length(jobj) do
13.       v ← jobj[i]
14.       field_table[v] ← field_table[v] + 1
15.     end for
16.   end for
17. end for
18. pattern_index ← 0
19. for header ∈ consistency_table do
20.   field_table ← consistency_table[header]
21.   patterns ← gen_patterns(header, field_table)
22.   save_to_file(patterns)
23.   msg_type ← extract_msg_type(patterns)
24.   if msg_type ≠ null then
25.     msg_types_set ← msg_types_set ∪ msg_type
26.     protocol_patterns ← protocol_patterns ∪ patterns
27.   end if
28. end for
```

算法1以待测协议名称protocol_name作为输入，输出为所提取的协议消息类型集合msg_types_set。第1行通过构造一致性提示词ConsPrompt（protocol_name）来初始化模板抽取请求。第2~17行采用多轮迭代以提升模板生成的多样性与稳定性。在每一轮迭代中，首先调用ChatLLM函数生成基础语法响应templates_ans（第4行），并进一步利用补充提示RemPrompt（protocol_name）生成扩展答案remaining_ans（第5、6行），合并结果形成候选模板集合comb_templates（第7行）。然后，调用extract_grammars（第8行）将部分模板解析为结构化语法对象grammar_list，并基于提取的头字段header与对应字段序列v构建一致性统计表consistency_table（第9~16行）。该统计过程通过记录字段出现频次，识别协议语法中的稳定共现结构。在第18~28行进入语法模板归纳阶段，对每个header，利用gen_patterns函数结合字段统计信息生成对应的正则模板patterns（第21行），再将其写入文件并保存（第22行）。通过extract_msg_type函数提取语法中的消息类型标识msg_type（第23行），并加入消息类型集合msg_types_set（第25行），同时将协议语法的正则化模板保存到protocol_patterns中（第26行）。最后，msg_types_set汇总了协议中可能出现的所有消息类型。

2.3 基于历史漏洞的种子富集

在协议模糊测试中，初始种子的质量在很大程度上决定了模糊测试的覆盖深度与漏洞发现效率。传统的种子生成方式多依赖人工构造、历史抓包数据或随机采样，这些方法存在协议语义覆盖不足、状态遍历不全和缺乏高风险触发特征等问题[33,34]。为此，本文设计了一种基于历史漏洞特征的种子富集机制，利用LLMs的语言生成能力，生成具备攻击潜力的高价值初始测试用例。该机制的核心思想在于利用公开漏洞信息构建漏洞触发模式数据库，并以此为基础设计提示词，引导LLMs自动生成模拟这些历史攻击特征的初始种子。通过结合协议语法规则与攻击行为语义，该模块有效解决了传统模糊测试中语法合规性与漏洞相关性的矛盾。图4展示了初始种子富集模块整体流程。

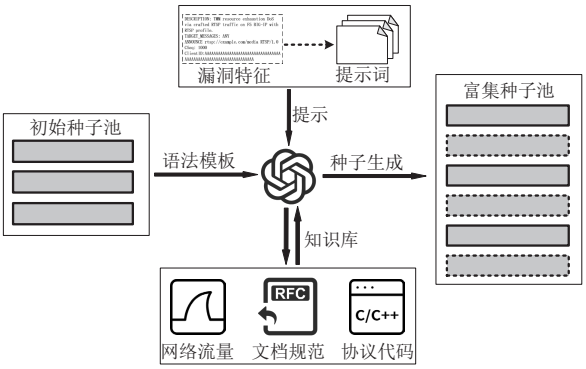


图 4 初始种子富集模块整体流程

初始种子富集模块整体流程主要包括漏洞触发模式的提取与建模、上下文感知的提示词生成以及序列生成与逻辑一致性检查3个阶段。通过引入结构化漏洞知识与提示工程技术，该模块能够引导LLMs生成既符合协议语法规范又带有历史漏洞特征的高价值种子。

1）漏洞触发模式的提取与建模

为了LLMs能够理解并利用历史漏洞信息，必须对其进行结构化提取与标准化建模。本文综合多种数据来源，包括国家信息安全漏洞共享平台（China National Vulnerability Database，CNVD）、CVE、安全厂商分析报告，

万方数据

以及主流协议实现的补丁与提交日志。针对每个漏洞样本，提取以下3项关键信息要素：（1）漏洞类型，如缓冲区溢出、特殊字符串、命令注入等；（2）适用范围，包括受影响的协议名称、版本信息、特定实现环境；（3）触发条件，涵盖攻击消息的类型（如FTP中的CWD、HTTP中的Range字段）、易受攻击字段、输入模式（如超长路径或特殊字符），以及必要的前置状态上下文。部分漏洞仅在满足特定会话历史交互后才会被触发，因此必须精确记录所需的请求顺序与上下文依赖。所有提取结果统一存储于结构化漏洞知识库中，作为后续提示工程与用例生成的重要输入。

2）上下文感知的提示词生成

为指导LLMs高效生成种子序列，本文提出了一种上下文感知的提示词构造方法。该方法根据现有种子序列的协议消息分布与语义结构来识别请求消息类型，并结合漏洞知识库中对应协议的漏洞特征构建提示词，指导模型生成既符合协议规范又具备攻击潜力的测试用例，如图5所示。提示词构造流程分为以下3个步骤：（1）解析原始种子序列，并与协议模板库中的完整消息集合做差集，识别出尚未出现但已在模板中定义的请求消息类型。将这些消息类型作为插入目标，用以扩展对未覆盖协议状态和代码路径的探索。（2）组织提示词内容，包括任务指令、上下文序列、插入目标类型列表、历史漏洞特征及输出格式。（3）将构造好的提示词输入LLMs，引导其生成语义连贯、逻辑完备的消息序列。

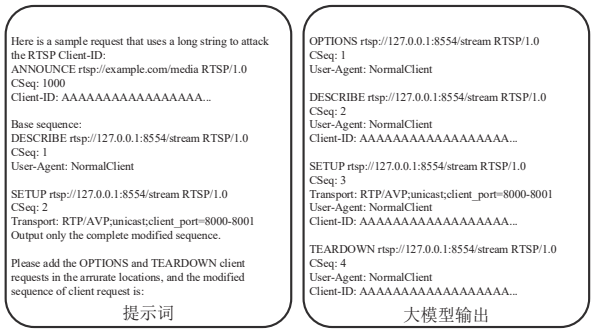


图 5 基于漏洞特征的种子富集提示词和 LLMs 输出

3）序列生成与逻辑一致性检查

有状态协议的交互序列之间通常受限于严格的状态转移关系，请求消息的插入位置必须遵循这种状态转换规则，否则极易生成语法合规但语义无效的测试用例。为确保生成序列的逻辑正确性，本文在提示词设计中嵌入协议上下文、历史请求序列及限定语句（如“The correct client request sequence is:”），引导LLMs推理出合理的请求插入位置。以RTSP协议为例，典型的合法请求序列为{DESCRIBE, SETUP, PLAY}。当插入新的请求类型（如RECORD）时，若位置不当（如置于PLAY之前），将导致状态机异常。通过提供历史上下文与规范示例，模型能够自动调整请求顺序，必要时插入过渡消息（如PAUSE），从而生成合法序列{DESCRIBE, SETUP, RECORD, PAUSE, PLAY}。此外，系统还引入了基于规则与有限状态机的验证模块，对生成序列进行自动筛选，剔除逻辑矛盾和不满足协议状态转移约束的用例，确保最终富集种子具备结构合法性与状态可达性。

2.4 结构感知的变异策略优化

传统模糊测试工具通常采用字节级的随机变异策略，尽管具备较强的通用性，但在网络协议测试场景下却容易破坏消息的结构完整性，导致生成的用例在早期解析阶段即被目标程序拒绝，从而严重浪费计算资源与测试时间。为此，本文提出了一种基于结构感知的变异优化策略，借助LLMs对协议语法的理解能力，在保证语法合规性的前提下，实现对关键字段的定向变异操作。该策略的核心思想在于充分利用第2.2节中构建的协议语法模板库，在进行变异前对输入消息进行语法解析，并将其划分为固定字段和可变字段两类。其中，固定字段（如请求类型、协议版本等）为协议结构的关键组成部分，禁止任何修改；可变字段包括参数值、可选头字段、标识符和路径信息等，可以进行有策略的变异操作。

变异流程如算法2所示。首先，模糊测试器将待变异的消息序列M分割为历史上下文M_1、当前变异目

万方数据

标消息M_2及剩余消息M_3。为了判断当前测试是否陷入了状态空间或代码覆盖的探索瓶颈，LPF引入了一个关键指标——平台期长度*PlateauLen*。该指标表示在当前协议状态下，模糊测试器连续生成的、未带来覆盖率提升或程序崩溃的“非感兴趣”输入数量，旨在量化测试过程是否进入“覆盖率平台期”。测试器每遇到一个未触发新路径或新崩溃的输入，就将*PlateauLen*加一，一旦出现新的路径或崩溃，*PlateauLen*则被重置。当*PlateauLen*未超过最大容忍阈值*Max*时，系统将以设定概率ε随机选择是否对M_2执行结构感知变异或传统随机变异。若*PlateauLen*达到上限*Max*，说明测试器可能卡在某一状态，传统变异难以继续推进，此时系统调用LLMs生成新的消息M_2'，以突破当前状态的限制并探索新的状态路径。最终，三段子序列合并为新的消息序列M'，用于后续执行。

算法2 结构感知变异算法

输入：待变异的消息序列M
输出：新的消息序列M′
1. StructureAwareMutate(M, G, R, PlateauLen, Max):
2. $(M_1, M_2, M_3) \leftarrow$ split_sequence(M)
3. if PlateauLen < Max:
4. if random() < ε:
5. $M_2' \leftarrow$ GrammarMutate(M_2, G)
6. else:
7. $M_2' \leftarrow$ RandomMutate(M_2)
8. else:
9. $M_2' \leftarrow$ ChatNextMsg(M_2, R)
10. PlateauLen $\leftarrow$ 0
11. $M' \leftarrow$ concatenate(M_1, M_2', M_3)
12. return M′, PlateauLen

在语法匹配阶段，系统维护一个映射结构$G=\{type \rightarrow grammar\}$，用于将消息类型（如RTSP协议中的PLAY、SETUP等）与对应的正则语法模板关联。该语法模板预先由LLMs基于少样本提示生成，结构上定义了字段标签与数据格式，所有可变异区域均使用<Value>占位符标注。模糊测试器运行时通过正则匹配对输入消息进行语法解析，提取结构化键值对并识别出所有可变异字段。如图6所示，变异阶段仅对标注为<Value>的字段执行插入、替换或删减等操作，同时确保变异结果不破坏其基本格式和语法规则。例如，对于Range字段的值，系统可注入超长字符串、边界值或非法时间片段，模拟缓冲区溢出或协议状态机混乱等异常场景。

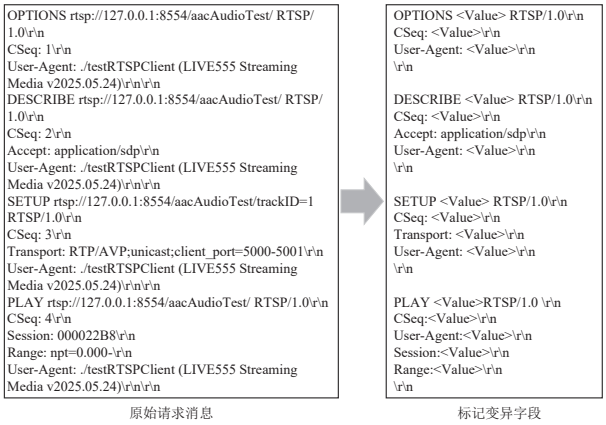


图6 结构感知变异示意图

为了兼顾种子执行效率与代码分支探索能力，系统引入混合策略，即在每轮变异中以设定概率交替使用结构感知变异与传统无结构变异。其中，结构感知策略更倾向生成高质量、语法合法的输入，提升对状态机深层路径的探索能力；无结构随机变异则保持了广度搜索能力，有效覆盖未建模或非典型路径。值得注意的是，所有语法模板在模糊测试开始阶段仅需提取一次，后续可复用于快速匹配与字段识别，从而实现语法信息与变异逻辑的解耦。这一设计不仅显著减少了LLMs的调用频率，提升了系统整体吞吐率，也增强了变异模块的可控性与可扩展性。对于无法匹配任何已知模板的输入，系统将自动回退至传统变异模式，以保证测试过程的连续性与兼容性。

3 实验与评估

3.1 实验环境

本文实验在一台配备Intel(R)Core(TM)i9-7960X CPU的机器上进行。这台机器具有32个逻辑核心，主频为2.80GHz，主内存为32GB，操作系统及版本为Ubuntu22.04.4 LTS。实验所选测试对象来自ProFuzzBench测试集，这是目前广泛认可的网络协议模糊测试基准套

万方数据

件，涵盖了多个常见文本协议的开源实现。被测协议及其信息如表1所示，涉及5种主流协议及其代表性实现。

表 1 被测协议及其信息

服务器软件	协议名称	版本号
ProFTPD	FTP	6903c5e
Live555	RTSP	2c92a57
Lighttpd1	HTTP	51d48de
Exim	SMTP	e3c7712
Kamailio	SIP	8d482b1

上述协议实现涵盖了文件传输、流媒体播放、网页服务、邮件传输和会话发起等典型网络交互场景，具有代表性强、覆盖协议语法复杂度广的特点。为便于性能评估与覆盖率分析，所有目标程序均使用GCC 9.4.0编译，并在编译阶段插入了代码覆盖追踪代码。

3.2 实验设计

为全面评估LPF在协议模糊测试中的有效性，本文选取了当前主流的两种代表性协议灰盒模糊测试工具（AFLNet与StateAFL）作为对比基线。AFLNet通过结合状态反馈与代码覆盖率反馈，引导测试路径优化；而StateAFL在此基础上进一步引入内存状态监测机制，以增强对复杂协议状态空间的探索能力。

实验从代码覆盖率、状态覆盖率及测试效率3个维度对LPF的性能进行系统性评估。覆盖率统计基于ProFuzzBench的自动化分析工具，分别测量各工具在不同协议实现上的代码与状态空间覆盖能力。为增强结果的稳定性，每项测试均独立重复3次，每次持续12 h，并对多轮测试结果取平均值进行比较分析。所有触发程序崩溃的测试用例均通过AFLNet-replay工具进行了复现，以确保其有效性和可重现性。在测试效率指标方面，本文以测试用例执行速率和首次崩溃触发时间为衡量指标。

整体实验流程分为初始化、测试执行与结果分析3个阶段。在初始化阶段中，调用LPF的语法提取模块，为每种目标协议构建语法模板库，并基于历史漏洞特征对原始种子集合进行富集。在测试执行阶段中，LPF与两种基线工具并行运行，系统自动记录代码覆盖与异常触发事件。在此阶段，LPF引入了关键参数——平台期长度阈值*Max*，用以调控LLMs的智能干预时机。当某状态连续*Max*次变异均无覆盖增益时，则判定为进入“平台期”，并由LLMs生成新消息以尝试突破。基于初步试验，本文将该值设定为300，以平衡“探索深度”与“测试效率”。若*Max*过小可更快触发LLMs，但会增加调用开销；*Max*过大虽有利于保持吞吐率，但会延长停滞期。在结果分析阶段，所有崩溃样本均经人工复现与核验，以确认触发机制与漏洞性质。

3.3 实验结果与分析

实验结果表明，LPF在协议语法提取准确率、代码覆盖率、状态覆盖率及测试效率等多个关键性能指标上，均优于现有的协议模糊测试工具。尤其是在协议语法自动化建模方面，LPF展现出优越的泛化能力和较高的准确性，为后续的种子变异阶段提供了更精准的指导。

3.3.1 协议语法提取准确率

在协议语法提取阶段，LPF基于LLMs自动生成协议语法模板。为验证LPF对不同LLMs的兼容性与泛化性，本文选取GPT-4、Qwen3和Code Llama三种主流LLMs进行实验验证，分别执行多轮RTSP协议的语法提取任务，采用相同的提示模板、temperature等设置，实验结果记录在表2中。以RTSP协议为例，通过30轮语法生成测试发现，LLMs生成了RFC标准中定义的全部10种请求消息类型。其中，GPT-4取得了最高的准确率，达到96.7%，Qwen3-235B准确率达94.8%，另一个代码专用模型Code Llama-34B准确率为91.2%。该结果验证了LPF在无需人工规则配置的情况下，能够高效、准确地还原复杂协议的语法结构。相较于传统的基于手工规则或静态语法描述语言（如Peach中的XML模板）方法，LPF在自动化程度、适应性和测试效率方面具备明显优势。同时，其输出语法结构在覆盖面和逻辑一致性上表现良好，为后续的结构感知变异提供了准确的语法指导。

万方数据

表 2 不同 LLMs 的协议语法提取准确率对比

大模型	参数规模 /B	准确率	缺失率	超出率
GPT-4	1760	96.7%	3.3%	4.7%
Qwen3	235	94.8%	5.2%	7.9%
Code Llama	34	91.2%	8.8%	10.4%

图 7 展示了 LLMs 生成的 RTSP 协议请求类型分布情况。进一步的误差分析表明，LLMs 在生成过程中主要存在以下 3 类语法偏差：1）语法缺失。部分标准请求类型未被正确生成，例如，“ANNOUNCE”和“GET_PARAMETER”在 30 轮测试中分别缺失 4 次和 2 次。2）幻觉生成。模型凭空构造了不存在于标准中的请求类型，如“SET_PUSH”。3）语法复合或重构。生成的请求语法结构虽具逻辑合理性，但偏离标准定义，或不具备明确语义功能，如“GET_DESCRIPTION”。另外，GPT-4 生成了 12 次“REDIRECT”请求，该请求类型虽然属于协议标准定义，但在大多数实际部署中并不具备可执行功能。针对这些问题，LPF 通过引入一致性阈值策略与多轮验证机制，剔除异常输出，确保最终生成模板的完整性与准确性。

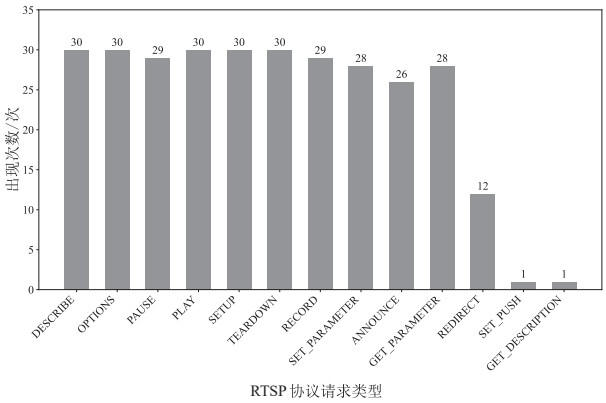


图 7 LLMs 生成的 RTSP 协议请求类型分布情况

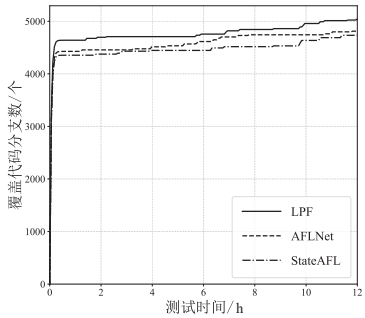
3.3.2 代码覆盖率

表 3 展示了 LPF、AFLNet 和 StateAFL 在代码覆盖率方面的对比结果，其中，提升率的计算方式为(LPF 覆盖数-基准工具覆盖数)/基准工具覆盖数×100%。本文统计了 3 次 12 h 测试中各工具所覆盖的平均代码分支数量，LPF 整体上优于两个基准工具。平均来看，在相同的运行时间内，LPF 比 AFLNet 多覆盖了 4.59% 的代码分支，比 StateAFL 多覆盖了 5.44% 的代码分支，表明 LPF 在处理各种协议时能更高效地覆盖代码分支。

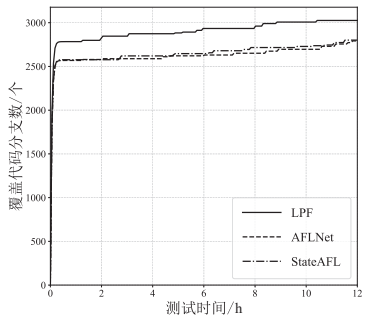
表 3 LPF 与基准工具的代码覆盖率对比

测试对象	LPF	AFLNet		StateAFL	
	覆盖分支数 / 个	覆盖分支数 / 个	提升率	覆盖分支数 / 个	提升率
ProFTPD	5043	4812	4.80%	4733	6.55%
Live555	3026	2792	8.38%	2801	8.03%
Lighttpd1	4726	4610	2.52%	4778	–1.09%
Exim	3880	3694	5.04%	3564	8.87%
Kamailio	9775	9565	2.20%	9323	4.85%

从各测试对象来看，LPF 在 Live555 协议上的提升最为显著，覆盖分支数量达到 3026 个，分别较 AFLNet 与 StateAFL 提升了 8.38% 和 8.03%；而在 Lighttpd1 服务器测试中 LPF 的性能提升相对有限，覆盖了 4726 个分支，仅比 AFLNet 多 2.52%，相较于 StateAFL 则下降了 1.09%。这主要是因为 Lighttpd1 的“单进程异步架构”逻辑相对集中，代码路径较为扁平。StateAFL 的随机变异策略可以充分探索这些核心路径，而 LPF 基于语法结构的变异策略在此类架构下难以发挥优势。图 8 展示了 3 种测试工具在不同协议上的代码覆盖率随测试时间的变化趋势。这些数据表明，LPF 在不同协议上的性能提升有所差异，但在大多数情况下能够显著提升代码分支覆盖率。

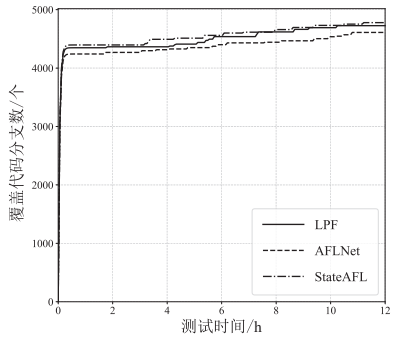


a）ProFTPD

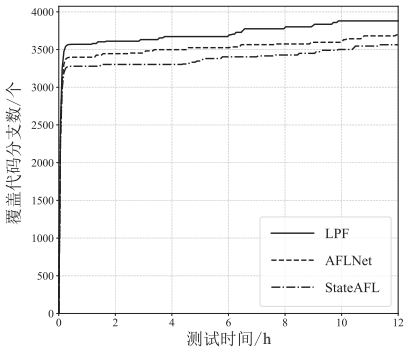


b）Live555

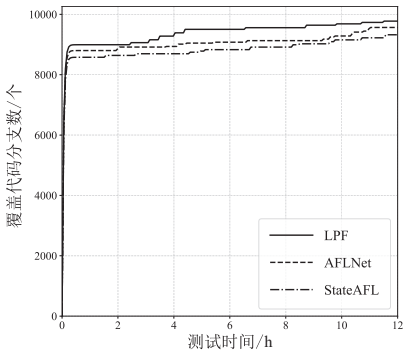
万方数据



c）Lighttpd1



d）Exim



e）Kamailio

图 8 3 种测试工具代码覆盖率变化趋势

3.3.3 状态覆盖率

为了评估LPF在协议状态空间探索方面的能力，本文对LPF与基准工具AFLNet和StateAFL进行了3轮、每轮持续12 h的状态覆盖实验，记录各测试工具在不同协议下平均覆盖的状态数量，实验结果如表4所示，其中，提升率的计算方式为(LPF覆盖数-基准工具覆盖数)/基准工具覆盖数×100%。LPF在大多数协议上均实现了更高的状态覆盖率，展现出其在状态感知引导机制方面的优势。在Live555协议中，LPF平均覆盖11.3个状态，较AFLNet提升28.41%，较StateAFL提升2.73%；在Exim协议中，LPF覆盖14.4个状态，同样超过AFLNet的11.0个状态和StateAFL的12.1个状态，状态覆盖率分别提升30.91%和19.01%；此外，在ProFTPD和Lighttpd1 协议中，LPF的状态覆盖提升幅度在10.85%~18.06%之间。然而，在对Kamailio的状态覆盖率测试中，LPF相较于StateAFL下降了13.68%。这主要是SIP协议本身的高度复杂性以及Kamailio的动态状态转移机制导致的。LLMs生成的语法模板难以完整捕捉这些动态路径，而StateAFL有一定概率探索到此类动态转移，从而表现更优。整体结果表明，LPF通过引入语义驱动的状态识别机制与结构感知变异策略，有效增强了对协议状态机的探索深度与广度，进一步提升了模糊测试的全面性和有效性。

表 4 LPF 与基准工具的状态覆盖率对比

测试对象	LPF	AFLNet		StateAFL	
	覆盖状态数/个	覆盖状态数/个	提升率	覆盖状态数/个	提升率
ProFTPD	23.5	20.4	15.20%	21.2	10.85%
Live555	11.3	8.8	28.41%	11.0	2.73%
Lighttpd1	8.5	7.2	18.06%	8.7	–2.30%
Exim	14.4	11.0	30.91%	12.1	19.01%
Kamailio	10.1	9.5	6.32%	11.7	–13.68%

3.3.4 测试效率

在测试效率方面，LPF在对5种协议软件的12 h测试周期内共触发了9次程序崩溃，明显优于AFLNet的4次和StateAFL的7次。进一步分析发现，LPF生成的测试用例有效融合了历史漏洞的攻击模式。例如，在SMTP协议的测试中，LPF引入了SQL注入类载荷，成功触发了多个独特的崩溃点，而AFLNet和StateAFL由于其变异策略的随机性与盲目性，未能覆盖这些关键执行路径。

表5 给出了不同工具首次触发程序崩溃所需时间的对比结果。LPF在ProFTPD、Live555和Lighttpd1三项测试中展现出更快的崩溃触发能力。在Live555测试中，LPF在2213.6 s时首次触发崩溃，分别优于AFLNet（2427.5 s）和StateAFL（4618.7 s），表明其生成的测试样本更具有针对性。在Lighttpd1测试中，LPF同样优于StateAFL，首次触发崩溃仅用时4845.4 s，而

万方数据

AFLNet在该协议下未能触发崩溃。在Exim测试中，尽管StateAFL的崩溃时间最短（594.9 s），但LPF也以864.6 s表现出接近性能，且明显优于AFLNet（1880.0 s），说明其在以命令交互为核心的邮件协议测试中依然具有竞争力。值得注意的是，在ProFTPD测试中，AFLNet和StateAFL在12 h内均未引发崩溃，而LPF在5845.4 s时成功触发崩溃，进一步凸显其在FTP协议测试中的有效性。

表5 不同工具首次触发程序崩溃所需时间的对比结果

测试对象	崩溃时间/s		
	AFLNet	StateAFL	LPF
ProFTPD	—	—	5845.4
Live555	2427.5	4618.7	2213.6
Lighttpd1	—	9228.3	4845.4
Exim	1880.0	594.9	864.6
Kamailio	1977.5	3250.6	1893.2

图9展示了3种测试工具在不同协议下的平均每秒执行次数对比，用以评估模糊测试的执行效率。在Lighttpd1和Live555测试中，LPF分别达到了28.1次/s和21.1次/s，显著高于其他两种工具；在Exim测试中，尽管整体执行频率较低，LPF仍以5.9次/s保持领先。相较而言，在Kamailio协议测试中，LPF（8.7次/s）的性能与AFLNet（9.1次/s）和StateAFL（8.5次/s）相近，三者差异较小；而在ProFTPD测试中，LPF的效率虽高于StateAFL，但略低于AFLNet。

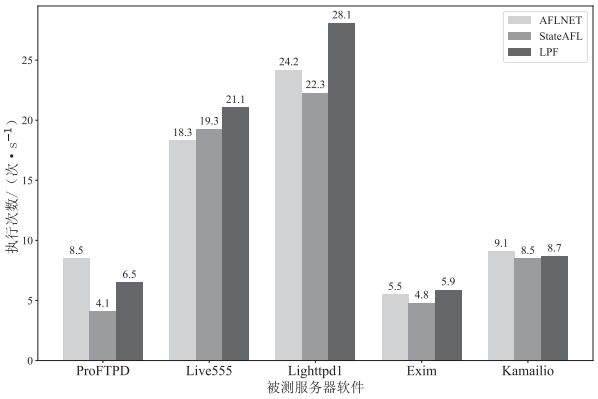


图9 3种测试工具平均执行次数对比

综上所述，LPF在大多数测试场景中展现出更高的执行效率，尤其是在Lighttpd1与Live555等复杂协议中，其种子生成机制与结构感知变异策略显著减少了冗余路径，从而提升了模糊测试的吞吐率。这些结果进一步验证了LPF在提升模糊测试效率方面的通用性与实用价值。

4 结束语

网络协议的安全性关乎整个网络空间的稳定运行与信任机制的构建。针对传统模糊测试在处理复杂协议语义和变异策略方面存在的局限性，本文提出并实现了一种融合LLMs的协议模糊测试框架LPF。该框架在提示词设计、协议语法抽取、历史漏洞知识迁移及结构感知变异策略等关键环节上实现了系统性改进，从而有效提升了模糊测试的智能化水平与实际测试能力。实验评估表明，LPF在多个典型协议场景中均表现出优异性能，在代码覆盖率、状态覆盖率及漏洞触发效率等方面显著优于其他代表性工具。

尽管LPF在验证语义驱动的协议模糊测试上展现出了综合优势，但当前框架在多语言协议实现的适配性、LLMs在极端场景下的输出稳定性以及对高度动态加密协议的精细化建模等方面仍有优化空间。未来的研究可从以下3个方向进一步拓展与深化。首先，可探索结合领域知识微调与上下文增强检索机制，利用协议RFC标准文档与公开漏洞样本数据构建高质量训练集，并通过冻结LLMs底层参数、仅微调上层适配层的方式，以较低成本注入协议领域知识，同时结合上下文增强检索机制，提升模型在跨协议环境下的泛化能力；然后，引入形式化方法对协议语法模板与状态建模结果进行自动验证，将LLMs生成的协议状态机模型转换为UPPAAL等工具的输入格式，在测试前执行可达性与安全属性检查，增强流程的准确性与鲁棒性；最后，结合符号执行与约束求解技术，探索LLMs与程序分析工具的协同推理机制，从而提升对深层协议逻辑路径与边界行为的探索能力。同时，将进一步构建更全面的评估框架，纳入更多最新协议模糊测试工具进行系统性对比，以更全面地验证LPF的适配性与优势。

万方数据

参考文献：

[1] BA J, BÖHME M, MIRZAMOMEN Z, et al. Stateful Greybox Fuzzing[C]//USENIX. The 31st USENIX Security Symposium. Berkely: USENIX Association, 2022: 3255−3272.

[2] LU Liyu, LIU Yuan, HONG Chao, et al. Screening Method of Fuzzy Test Seeds Based on Impact Orientation[J]. Network Security Technology & Application, 2024(2): 44−46.

陆力瑜,刘媛,洪超,等.基于影响性导向的模糊测试种子筛选方法[J]. 网络安全技术与应用，2024（2）：44−46.

[3] MILLER B P, ZHANG Mengxiao, HEYMANN E R. The Relevance of Classic Fuzz Testing: Have We Solved this One?[J]. IEEE Transactions on Software Engineering, 2022, 48(6): 2028−2039.

[4] MANÈS V J M, HAN H, HAN C, et al. The Art, Science, and Engineering of Fuzzing: A Survey[J]. IEEE Transactions on Software Engineering, 2021, 47(11): 2312−2331.

[5] ZHANG Xiaohan, ZHANG Cen, LI Xinghua, et al. A Survey of Protocol Fuzzing[J]. ACM Computing Surveys, 2024, 57(2): 1−36.

[6] ZHAO Yiru, GAO Long, WEI Qiang, et al. Towards Tightly−Coupled Hybrid Fuzzing via Excavating Input Specifications[J]. IEEE Transactions on Dependable and Secure Computing, 2024, 21(5): 4801−4814.

[7] PHAM V, BÖHME M, ROYCHOUDHURY A. AFLNet: A Greybox Fuzzer for Network Protocols[C]//IEEE. The 13th IEEE International Conference on Software Testing, Verification and Validation (ICST). New York: IEEE, 2020: 460−465.

[8] LUO Zhengxiong, YU Junze, DU Qingpeng, et al. Parallel Fuzzing of IoT Messaging Protocols through Collaborative Packet Generation[J]. IEEE Transactions on Computer−Aided Design of Integrated Circuits and Systems, 2024, 43(11): 3431−3442.

[9] MENG Ruijie, PHAM V−T, BÖHME M, et al. AFLNet Five Years Later: On Coverage−Guided Protocol Fuzzing[J]. IEEE Transactions on Software Engineering, 2025, 51(4): 960−974.

[10] Website. American Fuzzy Lop (AFL) Fuzzer[EB/OL]. [2025−07−30]. http://lcamtuf.coredump.cx/afl.

[11] LI Junqiang, LI Senyi, SUN Gang, et al. SNPSFuzzer: A Fast Greybox Fuzzer for Stateful Network Protocols Using Snapshots[J]. IEEE Transactions on Information Forensics and Security, 2022, 17: 2673−2687.

[12] ANDRONIDIS A, CADAR C. SnapFuzz: High−Throughput Fuzzing of Network Applications[C]//ACM. The 31st ACM SIGSOFT International Symposium on Software Testing and Analysis. New York: ACM, 2022: 340−351.

[13] QIN Shisong, HU Fan, MA Zheyu, et al. NSFuzz: Towards Efficient and State−Aware Network Service Fuzzing[J]. ACM Transactions on Software Engineering and Methodology, 2023, 32(6): 1−26.

[14] HUANG Tao, GAO Yansong, ZHENG Yifeng, et al. FineBID: Fine−Grained Protocol Reverse Engineering for Bit−Level Field Identification[J]. IEEE Transactions on Dependable and Secure Computing, 2025, 22(3): 2670−2686.

[15] KIM J, SEO M, MARIN E, et al. Ambusher: Exploring the Security of Distributed SDN Controllers through Protocol State Fuzzing[J]. IEEE Transactions on Information Forensics and Security, 2024, 19: 6264−6279.

[16] ZHANG Qingyu, LIN Jiayi, SUN Chenxin, et al. CherryPicker: A Parallel Solving and State Sharing Hybrid Fuzzing System[J]. IEEE Transactions on Dependable and Secure Computing, 2025, 22(4): 3324−3336.

[17] HONG Xuanquan, JIA Peng, LIU Jiayong. AFLNeTrans: Fuzzing of Protocols with State Relationship Awareness[J]. Netinfo Security, 2024, 24(1): 121−132.

洪玄泉，贾鹏，刘嘉勇. AFLNeTrans：状态间关系感知的网络协议模糊测试 [J]. 信息网络安全，2024，24（1）：121−132.

[18] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is All You Need[C]//ACM. The 31st International Conference on Neural Information Processing Systems (NIPS'17). New York: ACM, 2017: 6000−6010.

[19] BROWN T, MANN B, RYDER N, et al. Language Models are Few−Shot Learners[C]//ACM. The 34th International Conference on Neural Information Processing Systems (NIPS '20). New York: ACM, 2020: 1877−1901.

[20] ZHU Xiaogang, ZHOU Wei, HAN Q, et al. When Software Security Meets Large Language Models: A Survey[J]. IEEE/CAA Journal of Automatica Sinica, 2025, 12(2): 317−334.

[21] LEMIEUX C, INALA J, LAHIRI S, et al. Codamosa: Escaping Coverage Plateaus in Test Generation with Pre−Trained Large Language Models[C]//IEEE. The 45th IEEE/ACM International Conference on Software Engineering (ICSE). New York: IEEE, 2023: 919−931.

[22] DENG Yinlin, XIA C S, PENG Haoran, et al. Large Language Models are Zero−Shot Fuzzers: Fuzzing Deep−Learning Libraries via Large Language Models[C]//ACM. The 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis(ISSTA 2023). New York: ACM, 2023: 423−435.

[23] DENG Yinlin, XIA C S, YANG Chenyuan, et al. Large Language Models are Edge−Case Generators: Crafting Unusual Programs for Fuzzing Deep Learning Libraries[C]//ACM. The IEEE/ACM 46th International Conference on Software Engineering (ICSE'24). New York: ACM, 2024: 1−13.

[24] ZHANG Qiang, SHEN Yuheng, LIU Jianzhong, et al. ECG: Augmenting Embedded Operating System Fuzzing via LLM−Based Corpus Generation[J]. IEEE Transactions on Computer−Aided Design of Integrated Circuits and Systems, 2024, 43(11): 4238−4249.

[25] SHAHRIAR A, HISHAM S J, RAHMAN K M A, et al. 5GPT: 5G Vulnerability Detection by Combining Zero−Shot Capabilities of GPT−4 with Domain Aware Strategies through Prompt Engineering[J]. IEEE Transactions on Information Forensics and Security, 2025, 20: 7045−7060.

[26] ZHENG Tao, SHAO Jiang, DAI Jinqiao, et al. RESTLess: Enhancing State−of−the−Art REST API Fuzzing with LLMs in Cloud Service Computing[J]. IEEE Transactions on Services Computing, 2024, 17(6): 4225−4238.

[27] WANG Jincheng, YU Le, LUO Xiapu. LLMIF: Augmented Large Language Model for Fuzzing IoT Devices[C]//IEEE. The 2024 IEEE Symposium on Security and Privacy (SP). New York: IEEE, 2024: 881−896.

万方数据

[28] YANG Liqun, WEI Chaoren, YANG Jian, et al. Code Large Language Model-Based Fuzz Testing for Industrial IoT Programs[J]. IEEE Internet of Things Journal, 2024: 1-11.
[29] PIYUSH J, JOSEPH S, JAYA S G, et al. BertRLFuzzer: A BERT and Reinforcement Learning Based Fuzzer[C]//ACM. The Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence (AAAI'24/IAAI'24/EAAI'24). New York: ACM, 2024: 23521-23522.
[30] YANG Chenyuan, DENG Yinlin, LU Runyu, et al. WhiteFox: White-Box Compiler Fuzzing Empowered by Large Language Models[C]//ACM. The ACM on Programming Languages. New York: ACM, 2024, 8(2): 709-735.
[31] EOM J, JEONG S, KWON T. Fuzzing JavaScript Interpreters with Coverage-Guided Reinforcement Learning for LLM-Based Mutation[C]//ACM. The 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2024). New York: ACM, 2024: 1656-1668.
[32] XIA C S, PALTENGHI M, TIAN Jiale, et al. Fuzz4All: Universal Fuzzing with Large Language Models[C]//ACM. The 46th International Conference on Software Engineering. New York: ACM, 2024: 1-13.
[33] YE Kai, ZHU Xiaogang, XIAO Xi, et al. BazzAFL: Moving Fuzzing Campaigns towards Bugs via Grouping Bug-Oriented Seeds[J]. IEEE Transactions on Dependable and Secure Computing, 2025, 22(1): 179-191.
[34] LI Yuwei, JI Shouling, LYU Chenyang, et al. V-Fuzz: Vulnerability Prediction-Assisted Evolutionary Fuzzing for Binary Programs[J]. IEEE Transactions on Cybernetics, 2022, 52(5): 3745-3756.

万方数据